Optimizing the Order of Modes in Tensor Train Decomposition

Petr Tichavský, *Senior Member, IEEE*, and Ondřej Straka, *Member, IEEE*

***Abstract*—The tensor train (TT) is a popular way of representing high-dimensional hyper-rectangular data structures called tensors. It is widely used, for example, in quantum chemistry under the name "matrix product state". The complexity of the TT model mainly depends on the bond dimensions that connect TT cores, constituting the model. Unlike canonical polyadic decomposition, the TT model complexity may depend on the order of the modes/indices in the data structures or the order of the core tensors in the TT, in general. This letter aims to provide methods for optimizing the order of the modes to reduce the bond dimensions. Since the number of possible orderings of the cores is exponentially high, we propose a greedy algorithm that provides a suboptimal solution. We consider three problem setups, i.e., specifications of the tensor: tensor given by a list of all its elements, tensor described by a TT model with some default order of the modes, and tensor obtained by sampling a multivariate function.**

***Index Terms*—Tensors, tensor train decomposition, matrix product state.**

I. Introduction

MULTIDIMENSIONAL data occur in many applications, e.g., chemistry, astronomy, psychology, and many other fields. Such data are obtained when multivariate functions are sampled over a hyper-rectangular grid. The tensor train (TT) model is an alternative to the canonical polyadic decomposition that enables handling such data with a moderate number of parameters [1], [2]. In quantum chemistry, TT is known as the "matrix product state" and is used to model wave functions of quantum many-body systems [3]. The matrix product state is usually obtained by the density matrix renormalization group (DMRG) algorithm [4], [5]. This is a numerical variational technique aiming to minimize the total energy of the systems. Recently, TT's were shown to be useful in modeling the value function of the dynamic programming problem in active fault detection [6], [7].

The complexity of the TT model can be defined as the total number of its parameters. The number depends on the so-called bond dimensions or the size of the so-called TT cores, i.e., small-size tensors that constitute the model. Unlike the canonical polyadic decomposition, in TT, the complexity may depend on the order of the tensor dimensions or coordinates. TT is a special case of tensor tree networks [8]. Optimization of their topology is studied in [9]. The optimization is straightforward for tensors that fit the computer memory. For larger tensors, it relies on a bit problematic rank estimations of different matricizations of the tensors. In this letter, we propose a simpler method for a simpler model (TT).

The TT model of a tensor can be obtained in four ways. In quantum chemistry, it is obtained by a numerical optimization procedure. The other three ways directly work with the tensor elements. If all tensor elements are available and fit the computer memory, the best way is to find the TT model through multiple singular value decomposition (SVD) of a matrix. If some entries of the tensor are unknown (not available), one can resort to the alternating least square method [11], [12] or to a Krylov-Levenberg-Marquart (KLM) nonlinear optimization [13], [14]. Finally, if all tensor elements can be accessed but do not fit into memory, one can use a variant of the TT-cross algorithm [15], [16]. In this letter, we propose optimizing the order of the tensor modes for all three scenarios. Although we cannot prove the optimality of the algorithm, we show in simulations that with high probability, it finds the ordering with low model complexity.

The rest of the letter is organized as follows. Section II introduces the TT model; Section III offers a motivation for the optimization procedure. In Section IV, we present an SVD-based TT decomposition for tensors in both full and TT formats. In Section V, the optimization for the TT decomposition algorithms is proposed for both scenarios. Section VI contains computer simulations, and Section VII concludes the letter.

II. Tensor Train Model

Consider a general order-N tensor $\mathcal{T}$ with elements $T(i_1,\dots,i_N)$, $i_n{=}1,\dots,I_n$, $n{=}1,\dots,N$. A TT is a model of a tensor with a moderate number of parameters that are arranged in so-called TT cores or wagons. The wagons are order-3 tensors, except the first and last wagon, which are order-2, i.e., matrices. Each wagon is associated with a coordinate. The TT is given as a contraction of these wagons, see Fig. 1. Elements of the tensor train are computed as

$$T(i_1,\dots,i_N)=\sum_{b_1=1}^{r_1}\sum_{b_2=1}^{r_2}\cdots\sum_{b_{N-1}=1}^{r_{N-1}} \mathbf{w}_1(i_1,b_1)\mathbf{w}_2(b_1,i_2,b_2)\dots\mathbf{w}_N(b_{N-1},i_N).$$

Received 17 January 2025; revised 7 March 2025; accepted 9 March 2025. Date of publication 17 March 2025; date of current version 28 March 2025. This work was supported by Czech Science Foundation under Grant 22-11101S. The associate editor coordinating the review of this article and approving it for publication was Prof. Fangqing Wen.*(Corresponding author: Petr Tichavský.)*

Petr Tichavský is with the Czech Academy of Sciences, Institute of Information Theory and Automation, 18200 Prague, Czech Republic (e-mail: tichavsk@utia.cas.cz).

Ondřej Straka is with the Department of Cybernetics, University of West Bohemia, 30614 Bohemia, Czech Republic (e-mail: straka30@kky.zcu.cz).

Digital Object Identifier 10.1109/LSP.2025.3552005

1070-9908 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on October 21,2026 at 23:18:28 UTC from IEEE Xplore. Restrictions apply.

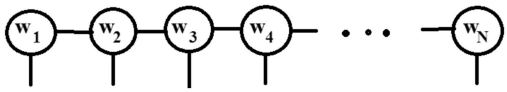


Fig. 1. Tensor train $\mathcal{T} = \{\{\mathbf{w}\}\}$ with cores (wagons) $\mathbf{w}_1, \ldots, \mathbf{w}_N$.

The integers $r_1, \ldots, r_{N-1}$ are called bond dimensions. The space/memory complexity of the model can be measured by the number of its parameters, that is

$$\#parameters = \sum_{i=1}^{N} r_{i-1} I_i r_i, \tag{1}$$

where $r_0 = r_N = 1$. A continuous extension of the TT is the functional decomposition [10],

$$V(\boldsymbol{\xi}) = \mathbf{W}_1(x_1)\mathbf{W}_2(x_2)\ldots\mathbf{W}_N(x_N), \tag{2}$$

where $\boldsymbol{\xi}$=$[x_1, \ldots, x_n]$ and $\mathbf{W}_n(x_n)$, n=$1, \ldots N$ are matrix-valued functions of scalars variables x_n for n=$1, \ldots, N$. The bond dimensions refer to sizes of the matrices; $\mathbf{W}_n(\cdot)$ has the size $r_{n-1} \times r_n$ for n=$1, \ldots, N$.

III. Motivation

Consider a special case of the tensor obtained by sampling the quadratic function

$$V_N(\boldsymbol{\xi}) = \boldsymbol{\xi}^T \mathbf{Q} \boldsymbol{\xi} \tag{3}$$

where $\mathbf{Q}$ is a symmetric square matrix of the size $N \times N$. For the construction of TT models of (3) for a general $\mathbf{Q}$, see Supplementary Materials or [20], [21]. Let N=5. We have shown in [18] that if the sampling grid is sufficiently dense, the bond dimensions of a TT representation of the tensor are at most 1,3,4,4,3,1 for a general $\mathbf{Q}$. For particular matrices $\mathbf{Q}$, the bond dimensions can be smaller. Thus, the specific bond dimensions mentioned here are, strictly speaking, upper bounds. Note that, throughout this section, the referred bond dimensions were obtained experimentally.

If $\mathbf{Q}$ is diagonal, $\mathbf{Q}$=diag$(Q_{11}, Q_{22}, \ldots, Q_{55})$, the bond dimensions are 1, 2, 2, 2, 2, 1, because in that case

$$V_5(\boldsymbol{\xi}) = \begin{bmatrix} 1 & Q_{11}x_1^2 \end{bmatrix} \begin{bmatrix} 1 & Q_{22}x_2^2 \\ 0 & 1 \end{bmatrix} \cdots \begin{bmatrix} 1 & Q_{44}x_4^2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} Q_{55}x_5^2 \\ 1 \end{bmatrix}. \tag{4}$$

Here, the bond dimensions do not depend on the order of the modes. If, however, $\mathbf{Q}$ is block diagonal, the situation is different. Let $\mathbf{Q}$ consist of two blocks $\mathbf{Q}_1$ and $\mathbf{Q}_2$ of the size 2×2 and 3×3, respectively, along its main diagonal,

$$\mathbf{Q} = \begin{bmatrix} Q_{11} & Q_{12} & 0 & 0 & 0 \\ Q_{21} & Q_{22} & 0 & 0 & 0 \\ 0 & 0 & Q_{33} & Q_{34} & Q_{35} \\ 0 & 0 & Q_{43} & Q_{44} & Q_{45} \\ 0 & 0 & Q_{53} & Q_{54} & Q_{55} \end{bmatrix}.$$

The bond dimensions are 1, 3, 2, 3, 3, 1, lower or equal to the bond dimensions for the general (full) $\mathbf{Q}$.

The bond dimensions would change if the coordinates of the tensor are permuted. However, the bond dimensions remain the same if the coordinates associated with a block $\mathbf{Q}_1$ or $\mathbf{Q}_2$ are mutually permuted. For example, the tensors with permuted modes as (2, 1, 3, 4, 5) or (1, 2, 5, 4, 3) have the same bond dimensions, 1, 3, 2, 3, 3, 1, but the permutation (1, 3, 2, 4, 5) gives the bond dimensions 1, 3, 4, 3, 3, 1.

Note that the bond dimensions of the quadratic function with two diagonal blocks can be deduced from the decomposition

$$V_N(\boldsymbol{\xi}) = \boldsymbol{\xi}_K^T \mathbf{Q}_1 \boldsymbol{\xi}_K + \boldsymbol{\xi}_{\bar{K}}^T \mathbf{Q}_2 \boldsymbol{\xi}_{\bar{K}} \tag{5}$$

where $\boldsymbol{\xi}_K$=$(x_1, \ldots, x_K)$ and $\boldsymbol{\xi}_{\bar{K}}$=$(x_{K+1}, \ldots, x_N)$ and $\mathbf{Q}_1$ and $\mathbf{Q}_2$ are the diagonal blocks of $\mathbf{Q}$ of the sizes $K\times K$ and $(N-K) \times (N-K)$, respectively. The bond dimensions between the two blocks can be proven to be 2, as in the case of diagonal $\mathbf{Q}$ in (4). Similar results are obtained for $\mathbf{Q}$ of the size 20×20. If all elements of $\mathbf{Q}$ are randomly chosen, the bond dimensions of the corresponding function are 1, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 1. The maximum bond dimension is 12. The quadratic function is sampled on the grid $(-2 : 0.5 : 2)$, but the sampling does not affect the bond dimensions. Assume now the $\mathbf{Q}$ is block diagonal, with two blocks of the size 10×10. Then, the bond dimensions of the TT representation are $1, 3, 4, 5, 6, 7, 6, 5, 4, 3, 2, 3, 4, 5, 6, 7, 6, 5, 4, 3, 1$. The maximum bond dimension is 7. These bond dimensions are obtained in the natural ordering of the modes, where the variables associated with the same block of $\mathbf{Q}$ are grouped together. If the ordering of the coordinates is different e.g., when the coordinates are mixed together, e.g., in the ordering (1, 11, 2, 12, 3, 13, 4, 14,....,10, 20), the TT representation has the maximum bond dimension 12 again. The question is how to find the proper ordering of the modes (dimensions of the tensor) in which the maximum bond dimension would be reduced back to 7.

Another interesting example is the tri-diagonal matrix $\mathbf{Q}$. It can be shown that in the natural ordering of the variables of the function (3), the upper bounds on the bond dimensions are $1, 3, \ldots, 3, 1$. However, in the "random" ordering of the variables, the bond dimensions can be the same as for a full $\mathbf{Q}$. The challenge is to find the modes ordering where the bond dimensions will be $1, 3, \ldots, 3, 1$ again.

Note that the same bond dimensions appear in the TT representation of the Rosenbrock function [14]. The latter function is not quadratic, but the relation between the variables is similar.

IV. SVD-Based TT Decomposition

The TT decomposition [1], [2] can be obtained through a series of *strangulations*, see [18]. The strangulation approximates a tensor $\mathcal{T}$ by a contraction of two smaller tensors. It can be accomplished by SVD of a matrix obtained by reshaping the tensor. The strangulation can be applied $N-1$ times to achieve the TT. Since SVD is a computationally cheap operation, the TT decomposition is easy to accomplish unless the tensor element number is too large.

The algorithm producing the TT model through SVD can be implemented in Algorithm 1 (in MATLAB notation).

In Algorithm 1, the bond dimension r_i between the wagons i and $i+1$ is the integer $I2$ in the line 5. The *bond value* is also worth attention and is defined as the $I2$-th singular value in $\mathbf{S}$,

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on October 21,2026 at 23:18:28 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: SVD-Based TT Decomposition.

```
1  function F = ttsvd(T,tol)
2  sz=size(T); N=length(sz); F=cell(1,N); I1=1; X=T;
3  for i = 1:N-1
4    [U,S,V] = svd(reshape(X,sz(i)*I1,[]));
5    I2 = sum(diag(S)>tol*S(1,1));
6    F{i} = reshape(U(:,1:I2),[I1,sz(i),I2]);
7    X = S(1:I2,1:I2) * V(:,1:I2)';
8    I1 = I2;
9  end
10 F{N} = reshape(X,I2,sz(N),1);
```

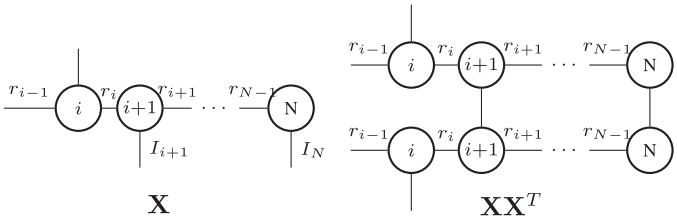


Fig. 2. Illustration of matrices $\mathbf{X}$ and $\mathbf{X}\mathbf{X}^T$.

$\mathbf{S}(I2, I2)$. It is the smallest singular value of the matrix $\mathbf{X}$ in step 4 that is greater than the tolerance limit "tol". If the bond value is close to zero, the bond dimension would be lower than $I2$ for a higher tolerance limit. It is important to note that the bond dimension and the bond value in step i of the algorithm do not depend on the order of the coordinates $i+1, \ldots, N$ because the singular values of the matrix $\mathbf{X}$ do not change if its columns are arbitrarily permuted.

Algorithm 1 assumes that all tensor elements are available and fit the computer memory. We show that Algorithm 1 can be rewritten for a tensor that does not fit into memory and is given using a TT model. The new Algorithm 2 still does not alter the order of the coordinates, but it allows the transformation of one TT model into another, possibly with lower bond dimensions. The tensor may not fit the computer memory in the full format.

Algorithm 2 is suitable for numerical operations with tensors already given in the TT model. For example, if tensor T is given as a sum of two tensors T_1 and T_2 in TT format with bond dimensions b_1 and b_2, it is possible to write the sum $T_1 + T_2$ in the form of TT with bond dimension $b_1 + b_2$, [19]. Algorithm 2 can help to reduce the latter bond dimension to a lower value. For example, if both T_1 and T_2 are quadratic with general $\mathbf{Q}$'s, cf. (3), then $T_1 + T_2$ is quadratic as well, and its bond dimensions should not be larger than $b_1 = b_2$.

The main idea of Algorithm 2 is to replace the SVD of the matrix $\mathbf{X}$ in line 4 of Algorithm 1 by the eigenvalue decomposition of the product $\mathbf{X}\mathbf{X}^T$. The matrix $\mathbf{X}$ has the size $(I_i r_i) \times (I_{i+1} \ldots I_N)$ where r_i is the bond dimension between the i-th wagon and the rest of the train. If the SVD of $\mathbf{X}$ is $\mathbf{U}\mathbf{S}\mathbf{V}^T = \mathbf{X}$, then the eigendecomposition (or SVD) of $\mathbf{X}\mathbf{X}^T$ is $\mathbf{X}\mathbf{X}^T = \mathbf{U}\mathbf{S}^2\mathbf{U}^T$. The later matrix has the size $(I_i r_i) \times (I_i r_i)$ and can be computed through tensor contractions; see Fig. 2. The second output of Algorithm 2 is a vector of the bond values.

V. Optimizing Order of the Tensor Modes

The ultimate complexity criterion for optimizing the modes' order can be the total number of parameters (1). Sometimes it

Algorithm 2: SVD-Based TT Decomposition in TT Format.

```
1  function [G,b] = ttsvdtt(F,tol)
2  G = F; I1 = 1; b = zeros(1,N-1);
3  for i = 1:N-1
4    xx = 1;
5    for j = N:-1:i+1
6      [r1,~,r2] = size(F{j});
7      xx = reshape(F{j},[],r2)*reshape(xx,r2,[]);
8      xx = reshape(xx,r1,[])*reshape(F{j},r1,[])';
9    end
10   [r1,sz,r2] = size(F{i});
11   xx=reshape(F{i},[],r2)*xx*reshape(F{i},[],r2)';
12   [U,S,~] = svd(xx);
13   sval = sqrt(diag(S));
14   I2 = sum(sval>tol*sval(1));
15   b(i) = sval(I2);
16   G{i} = reshape(U(:,1:I2),[I1,sz,I2]);
17   s = size(F{i+1}); s(1) = I2;
18   aux = U(:,1:I2)'*reshape(F{i},[],r2);
19   F{i+1} = reshape(aux*reshape(F{i+1},r2,[]),s);
20   I1 = I2;
21 end
22 G{N} = F{N};
23 end
```

Algorithm 3: Optimized SVD-Based TT Decomposition.

```
1  function [F,order] = ttsvdopt(T,tol)
2  sz = size(T); N = length(sz); F = cell(1,N);
3  I1 = 1; X = T ; order = 1:N;
4  for i = 1:N-1
5    [U,S,V] = svd(reshape(X,sz(i)*I1,[]));
6    I2 = sum(diag(S) > tol*S(1,1));
7    v = S(I2, I2); ord = order; Xa = X; sza = sz;
8    for j = i+1:N
9      Xa = reshape(Xa,I1,sza(j-1),[]);
10     Xa = permute(Xa,[1,3,2]);
11     Xa = reshape(Xa,sz(i)*I1,[]);
12     [Ua,Sa,Va] = svd(Xa);
13     I2a = sum(diag(Sa)>tol);
14     if I2a < I2||(I2a == I2 && Sa(I2,I2) < v)
15       U = Ua; S = Sa; V = Va;
16       I2 = I2a; v = S(I2,I2);
17       order = [ord(1:i-1),ord(j:N),ord(i:j-1)];
18       sz = [sza(1:i-1),sza(j:N),sza(i:j-1)];
19     end
20   end
21   F{i} = reshape(U(:,1:I2),[I1,sz(i),I2]);
22   X = S(1:I2,1:I2)*V(:,1:I2)'; I1 = I2;
23 end
24 F{N} = reshape(X,I2,sz(N),1)
```

can be reduced by permuting the tensor modes. Assume first that the whole tensor can fit the computer memory. The brute force optimization of the modes' ordering would be to check all $N!$ permutations of the modes through Algorithm 1 and select the solution with the smallest maximum bond dimension. The complexity of such an algorithm would be exponential in N.

Instead, we propose a suboptimal *greedy* algorithm that has the complexity of $O(N^2)$ applications of Algorithm 1 and appears to have a good performance. The algorithm begins with finding the mode that has the smallest bond dimension and the smallest bond value with respect to the remainder of the tensor. This mode is taken for the left-end wagon of the sought TT. The second mode in the new TT is sought as the one that has the lowest bond dimension and the smallest bond value with respect to the rest of the tensor, and so on. The algorithm produces the (sub-)optimum TT as a side product.

Assume now that the tensor is so huge in size that it is practically impossible to store all elements in computer memory and

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on October 21,2026 at 23:18:28 UTC from IEEE Xplore. Restrictions apply.

work with the full tensor. There are basically two possibilities for how the large-order tensors can be specified. First, the tensor can be obtained by sampling a multivariate function of N variables. It is easy to permute the input coordinates by manipulating the function arguments but much harder to permute the TT cores (wagons) of the corresponding TT. The second possibility is that the tensor is given as a TT with a non-optimal order of the TT cores (wagons), meaning not necessarily the lowest bond dimensions. Such a situation happens in quantum chemistry, for example.

The second possibility of presenting a tensor, i.e., as a TT model, can be cast to the first option. Once we have a tensor represented by a TT, we have access to all the tensor elements, and the TT can be viewed as a discrete function of N variables. We can apply the TT-cross algorithm to obtain a TT approximation of this function for the arbitrary ordering of the input coordinates.

The TT-cross decomposition is a TT approximation method that works with only a limited number of the tensor elements, cf. [15], [16], [17]. It exists in several variants, such as "greedy-cross", "dmrg-cross" or "amen-cross" [17]. The algorithms' input is a multivariate function. Applying the TT-cross algorithm to the function with all possible modes' orderings, we could do the optimization and select the best possible ordering. The problem is that their number $N!$ might be too large again.

We propose an analogy to Algorithm 3 for this scenario and denote it as Algorithm 4. Its input is a function computing the values of the tensor elements, denoted "getT", and tolerance constant "tol". If the tensor is given in the form of TT, then getT is defined through the function evalTT in the Supplementary materials, getT=@(ix) evalTT(W,ix). The output of the Algorithm 4 is a TT decomposition together with a permutation of the modes. The procedure utilizes one of the TT-cross algorithms, which is applied to the function "getT" for different permutations of the input coordinates. Once we get the TT candidate, we apply Algorithm 2 to get its bond values.

In each step, Algorithm 4 selects the next coordinate among the remaining modes as the one with the lowest bond dimension and bond value with respect to the remainder of the tensor. The procedure requires $O(N^2)$ runs of the TT-cross algorithm and "ttsvdtt", Algorithm 2.

VI. Simulation Examples

Since we do not provide formal proof that the proposed technique *always* finds the right modes' ordering, i.e., ordering of the wagons in the TT decomposition, we test it in simulations. We follow the example in Section III, considering a quadratic function of $N{=}20$ variables with symmetric $\mathbf{Q}$ that is (a) block diagonal with two blocks of the size $10{\times}10$ and (b) tridiagonal. The nonzero elements of $\mathbf{Q}$ below the main diagonal are generated as Gaussian i.i.d. with zero mean and variance one. The variables were sampled at the grid $(-2:0.5:2)$ so that the full tensor has 9^{20} elements. The variables (row and columns of $\mathbf{Q}$) are randomly permuted, and the quadratic form is decomposed through Algorithm 4 in 1000 independent trials. The results (counts of the maximum bond dimension in each model) are summarized in Tables I and II.We can see that the best ordering, i.e., the lowest bond dimensions, was achieved only in two cases of 1000. However, in all trials, a significant complexity reduction was obtained. On average, the complexity has dropped from the initial 10,206 to 5,673.9 parameters (nearly one-half). The minimum number of parameters was 3,906.

Algorithm 4: Optimized TT-Cross Decomposition.

```
function [F,order] = ttcrossopt(getT,sz,tol)
N = length(sz); order = 1:N;
F = tt_cross(sz,getT,tol);
[F,bval] = ttsvdtt(F,tol);
for i = 1:N-1
  b = size(F{i},3); ord = order;
  vmin = bval(i);
  for j = 1:N-i
    ord = [ord(1:i-1),ord(i+1:N),ord(i)];
    getTp = @(ix) getT(ord(ix));
    Fa=tt_cross(sz(ord),getTp,tol);
    [Fa,bvala] = ttsvdtt(Fa,tol);
    b2 = size(Fa{i},3); v2 = bvala(i);
    if b2 < b || (b2 == b && v2 < vmin)
      F = Fa; order = ord; b = b2; vmin = v2;
    end
  end
end
```

TABLE I
COUNTS OF THE MAXIMUM BOND DIMENSIONS FOR BLOCK-DIAGONAL $\mathbf{Q}$ IN 1000 INDEPENDENT TRIALS

max. bond dim.	7	8	9	10	11
#occurences	2	223	749	26	0

TABLE II
COUNTS OF THE MAXIMUM BOND DIMENSIONS FOR TRI-DIAGONAL $\mathbf{Q}$ AND FOR ROSENBROCK'S FUNCTION IN 1000 INDEPENDENT TRIALS

max. bond dim.	3	4	5	6	7
#occur's trilin. $\mathbf{Q}$	258	360	274	92	16
#occur's Ross.	1000	0	0	0	0

Similar results were obtained for the trilinear $\mathbf{Q}$, see Table II. Here, the optimum ordering was achieved in 25% trials. On average, the complexity dropped to 2,397.8 parameters (the minimum was 1,512). In the case of the Rosenbrock function with randomly ordered input parameters, the optimum ordering was found in 100% trials. Note that Algorithms 3 and 4 are effective when the TT approximates a tensor that has an inherent structure. Reordering the tensor modes can reduce the TT's bond dimensions. However, it is not possible to identify the structure solely based on the distribution characteristics of the TT bond dimensions before the reordering.

VII. Conclusion

In this letter, greedy techniques were proposed to provide a suboptimal mode ordering for TT model computation, minimizing model complexity. The proposed techniques were shown to perform well, meaning that with high probability, they can find the ordering with low complexity of the model. The future plan is to design an algorithm for optimizing the order of the wagons in TT without using the TT-cross algorithm.

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on October 21,2026 at 23:18:28 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] I. V. Oseledets and E. E. Tyrtyshnikov, "Breaking the curse of dimensionality, or how to use SVD in many dimensions," *SIAM J. Sci. Comput.*, vol. 31, no. 5, pp. 3744–3759, 2009.

[2] I. V. Oseledets, "Tensor–train decomposition," *SIAM J. Sci. Comput.*, vol. 33, no. 5, pp. 2295–2317, Jan. 2011.

[3] V. Khoromskaia and B. N. Khoromskij, *Tensor Numerical Methods in Quantum Chemistry*. Berlin, Germany: De Gruyter, 2018.

[4] S. R. White, "Density matrix formulation for quantum renormalization groups," *Phys. Rev. Lett.*, vol. 69, pp. 28–63, 1992.

[5] S. R. White, "Density-matrix algorithms for quantum renormalization groups," *Phys. Rev. B., Condens. Matter*, vol. 48, no. 14, Oct. 1993, Art. no. 10345.

[6] I. Punčochář, O. Straka, and P. Tichavský, "Active fault detection based on tensor train decomposition," *IFAC-PapersOnLine*, vol. 58, no. 4, pp. 676–681, 2024.

[7] P. Tichavský, O. Straka, and I. Punčochář, "Discrete-time dynamic programming using tensor trains," unpublished.

[8] L. Grasedyck and W. Hackbusch, "An introduction to hierarchical (H-) rank and TT-rank of tensors with examples," *Comput. Methods Appl. Math.*, vol. 11, pp. 291–304, 2011.

[9] J. Ballani and L. Grasedyck, "Tree adaptive approximation in the hierarchical tensor format," *SIAM J. Sci. Comput.*, vol. 36, no. 4, pp. A1415–A1431, 2014.

[10] A. Gorodetsky, S. Karaman, and Y. M. Marzouk, "A continuous analogue of the tensor-train decomposition," *Comput. Methods Appl. Mechan. Eng.*, vol. 347, no. 4, pp. 59–84, 2018, doi: 10.1016/j.cma.2018.12.015.

[11] S. Holtz, T. Rohwedder, and R. Schneider, "The alternating linear scheme for tensor optimization in the tensor train format," *SIAM J. Sci. Comput.*, vol. 34, no. 2, pp. A683–A713, 2012, doi: 10.1137/100818893.

[12] L. Grasedyck, M. Kluge, and S. Krämer, "Variants of alternating least squares tensor completion in the tensor train format," *SIAM J. Sci. Comput.*, vol. 37, no. 5, pp. A2424–A2450, 2015, doi: 10.1137/130942401.

[13] P. Tichavský, A. H. Phan, and A. Cichocki, "Krylov-Levenberg-Marquardt algorithm for structured Tucker tensor decompositions," *IEEE J. Sel. Topics Signal Process.*, vol. 15, no. 3, pp. 550–559, Apr. 2021.

[14] P. Tichavský and A. H. Phan, "Tensor chain decomposition and function interpolation," in *Proc. IEEE Stat. Signal Process. Workshop*, Hanoi, Vietnam, 2023, pp. 557–561.

[15] I. V. Oseledets and E. E. Tyrtyshnikov, "TT-cross approximation for multidimensional arrays," *Linear Algebra Appl.*, vol. 432, no. 1, pp. 70–88, 2010, doi: 10.1016/j.laa.2009.07.02.

[16] D. V. Savostyanov, "Quasioptimality of maximum-volume cross interpolation of tensors," *Linear Algebra its Appl.*, Vol. 458, pp. 217–244, 2014.

[17] I. Oseledets, "MATLAB toolbox for working with high-dimensional tensors in the tensor-train (TT)-format," Version 2.2 2013. [Online]. Available: https://github.com/oseledets/TT-Toolbox

[18] P. Tichavský and O. Straka, "Tensor train approximation of multivariate functions," in *Proc. IEEE 32nd Eur. Signal Process. Conf.*, Lyon, France, 2024, pp. 2262–2266.

[19] N. Lee and A. Cichocki, "Fundamental tensor operations for large-scale data analysis using tensor network formats," *Multidimens. Syst. Signal Process.*, vol. 29, no. 3, pp. 921–960, 2018.

[20] S. Dolgov and B. Khoromskij, "Two-level QTT-Tucker format for optimized tensor calculus," *SIAM J. Matrix Anal. Appl.*, vol. 34, no. 2, pp. 593–623, 2013.

[21] V. Kazeev, O. Reichmann, and C. Schwab, "Low-rank tensor structure of linear diffusion operators in the TT and QTT formats," *Linear Algebra its Appl.*, vol. 438, no. 11, pp. 4204–4221, 2013.

[22] P. Tichavský, "Optimizing order of modes in tensor trains," 2025. [Online]. Available: https://github.com/Tichavsky/tensor-decomposition

Authorized licensed use limited to: Indian Institute of Information Technology Dharwad. Downloaded on October 21,2026 at 23:18:28 UTC from IEEE Xplore. Restrictions apply.